```
var depthScale = d3.scale.category10([0,1,2]);

var packChart = d3.layout.pack();
packChart.size([500,500])
     .children(function(d) {
                 return d.values;
     })
     .value(function(d) {
                return 1;
     });

d3.select("svg")
        .selectAll("circle")
        .data(packChart(packableTweets))
        .enter()
        .append("circle")
        .attr("r", function(d) {return d.r;})
        .attr("cx", function(d) {return d.x;})
        .attr("cy", function(d) {return d.y;})
        .style("fill", function(d) {return depthScale(d.depth);})
        .style("stroke", "black")
        .style("stroke", "2px");
```

Creates a color scale to color each depth of the circle pack differently

Sets the size of the circle-packing chart to the size of our canvas

Creates a function that returns 1 when determining the size of leaf nodes

Sets the pack accessor function for child elements to look for "values", which matches the data created by d3.nest

Binds the results of packChart transforming packableTweets

Radius and xy coordinates are all computed by the pack layout

Gives each node a depth attribute that we can use to color them distinctly by depth

Notice that when the pack layout has a single child (as in the case of Sam, who only made one tweet), the size of the child node is the same as the size of the parent. This can visually seem like Sam is at the same hierarchical level as the other Twitter users who made more tweets. To correct this, we can modify the radius of the circle. That accounts for its depth in the hierarchy, which can act as a margin of sorts:

```
.attr("r", function(d) {return d.r - (d.depth * 10)})
```

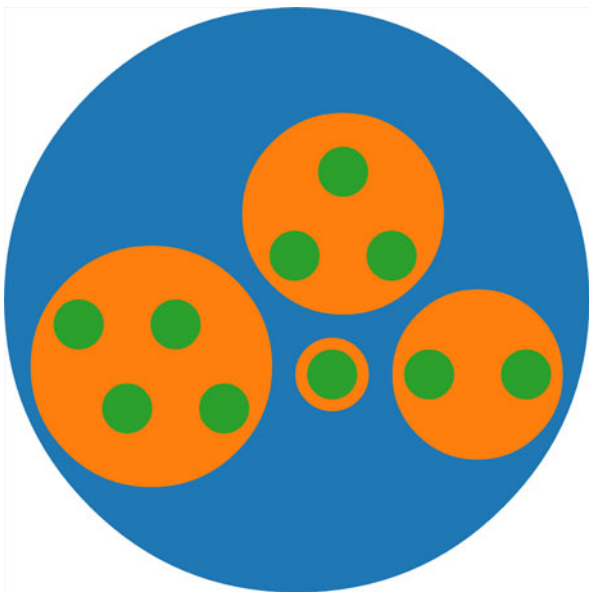


Figure 5.11 An example of a fixed margin based on hierarchical depth. We can create this by reducing the circle size of each node based on its computed "depth" value.